TECHNICAL ASSESSMENT SOFTWARE DEVELOPMENT ENGINEER (SDE) — Recruitment

Project Title: DealBridge — Product Discovery & Affiliate Commerce Platform

MERN Stack Developer — Fresher Recruitment

Project Title: DealBridge — Product Discovery & Affiliate Commerce Platform

Technology Stack: MongoDB, Express.js, React.js, Node.js

Assessment Duration: 4 Days (96 Hours)

Total Marks: 100

Deployment Requirement: Mandatory

Cost Constraint: Zero-cost implementation using free tools and services

OBJ

1. PROJECT OVERVIEW

Business Context

We are looking to build a product discovery and affiliate commerce platform that allows users to browse products across different categories and compare offers from external e-commerce platforms.

Unlike a traditional e-commerce website, our platform will not directly sell or maintain inventory of products. Instead, it will display product information and redirect users to external merchant websites, such as Amazon or Flipkart, when they choose to purchase a product.

For example, a user visits DealBridge and searches for a watch. The platform displays the watch's image, name, brand, specifications, and available merchant offers. When the user clicks “Buy Now,” they are redirected to the corresponding product page on the external merchant's website.

The objective is to develop a simplified but functional version of a real-world product discovery and affiliate commerce platform.

Important: This assessment does not require actual affiliate partnerships, paid APIs, or live merchant integrations. Candidates may use seeded product data and valid external product URLs.

2. PROJECT OBJECTIVES

The application should demonstrate the candidate's ability to:

Develop a full-stack application using the MERN stack.

Design and implement RESTful APIs.

Model and manage relationally connected data using MongoDB.

Implement authentication and role-based access control.

Develop product search, filtering, sorting, and pagination.

Associate multiple merchant offers with a single product.

Implement secure external redirects and click tracking.

Build a responsive and user-friendly interface.

Deploy a working application using free hosting options.

3. FUNCTIONAL REQUIREMENTS

Module 1: Product Discovery and Listing

Develop a responsive homepage displaying products across multiple categories, such as:

Watches

Smartphones

Laptops

Headphones

Fashion

Home appliances

Each product card should display:

Product image

Product name

Brand

Category

Starting price or lowest available offer price

Availability status, where applicable

A link to view product details

Search and filtering requirements:

Search products by name, brand, or model.

Filter products by category and brand.

Filter products by minimum and maximum price.

Sort products by price, newest arrivals, and popularity.

Implement pagination for product listings.

Search and filtering should operate on backend data rather than only filtering products already loaded into the browser.

Module 2: Product Details and Merchant Offers

Each product must have a dedicated details page containing:

Product name and description

Brand and model

Product images

Specifications or key features

Category

Available merchant offers

For example, the same watch may have offers from multiple external stores.

Each merchant offer should contain:

Merchant name

Product price

External product URL

Availability status, where applicable

Last-updated timestamp

Business rule: Multiple merchant offers should be associated with one canonical product whenever they represent the same product and variant.

The platform should identify and highlight the lowest available listed price.

Module 3: External Merchant Redirection

When a user clicks “Buy Now,” the application should redirect them to the configured external product page.

Requirements:

Each offer must have a valid destination URL.

The system should record the product, merchant, timestamp, and click event.

The click-tracking mechanism should not unnecessarily block the redirect.

Invalid or untrusted redirect destinations must be rejected.

Users must not be able to supply arbitrary redirect URLs through request parameters.

The platform does not need to process payments or manage orders.

Module 4: User Authentication and Wishlist

Implement user registration, login, and logout.

Authenticated users should be able to:

Add products to their wishlist.

Remove products from their wishlist.

View their saved products.

Retain wishlist data after logging out and logging in again.

Access control requirements:

Users must only be able to view and modify their own wishlist.

Authentication and authorization must be enforced on the backend.

Passwords must be securely hashed and must never be stored in plain text.

Module 5: Admin Dashboard

Create a basic administrative interface for managing the product catalog.

Administrators should be able to:

Add new products.

Edit existing product details.

Add or update merchant offers.

Change product availability or deactivate products.

View basic merchant click statistics.

Security requirement: Only authorized administrators may create, modify, or deactivate products and merchant offers. Hiding admin controls in the frontend is not sufficient; the backend must enforce permissions.

Module 6: Basic Analytics

Implement a simple dashboard displaying:

Total number of products

Total number of merchant offers

Total outbound merchant clicks

Most-clicked products

Most-clicked merchants

Analytics should be calculated using stored application data rather than hardcoded statistics.

OBJ

4. TECHNICAL REQUIREMENTS

Frontend

React.js

Responsive UI for desktop and mobile

React Router or an equivalent routing solution

Appropriate state management

API integration with the backend

Form validation and error handling

Loading, empty, and error states

Backend

Node.js and Express.js

RESTful API architecture

Authentication and authorization middleware

Request validation and centralized error handling

Pagination, filtering, and sorting

Appropriate HTTP status codes

Secure handling of user input and external URLs

Database

Use MongoDB to store application data.

Suggested collections:

Users

Products

Merchants

MerchantOffers

Wishlists

ClickEvents

Candidates may modify the schema or collection structure according to their implementation, provided the data model supports the required functionality.

Suggested API Endpoints

The following endpoints are indicative. Candidates may use a different RESTful structure if it is documented and implemented consistently.

Products

GET /api/products — List, search, filter, sort, and paginate products.

GET /api/products/:id — Retrieve product details.

POST /api/products — Create a product (admin only).

PUT /api/products/:id — Update a product (admin only).

DELETE /api/products/:id — Deactivate or remove a product (admin only).

Merchant Offers

GET /api/products/:id/offers — Retrieve offers for a product.

POST /api/products/:id/offers — Add an offer (admin only).

PUT /api/offers/:id — Update an offer (admin only).

Authentication and Wishlist

POST /api/auth/register — Register a user.

POST /api/auth/login — Authenticate a user.

GET /api/wishlist — Retrieve the authenticated user's wishlist.

POST /api/wishlist/:productId — Add a product to the wishlist.

DELETE /api/wishlist/:productId — Remove a product from the wishlist.

Analytics

GET /api/admin/analytics — Retrieve basic product and click statistics (admin only).

Candidates may add further endpoints as needed.

OBJ

5. FOUR-DAY IMPLEMENTATION PLAN

DAY 1 — Frontend and Product Discovery

Expected deliverables:

Set up the frontend and backend projects.

Design the homepage and product listing pages.

Implement product cards and product detail pages.

Implement search, filtering, sorting, and pagination interfaces.

Create an initial product dataset.

End-of-day target: Users can browse and search the product catalog through a functional interface.

DAY 2 — Backend, Database, and Merchant Offers

Expected deliverables:

Design the MongoDB schemas.

Implement REST APIs for products and merchant offers.

Connect the frontend to the backend.

Associate multiple merchant offers with a canonical product.

Implement lowest-price selection and server-side filtering.

Add validation and error handling.

End-of-day target: Product information and merchant offers are loaded from the database and displayed correctly.

DAY 3 — Authentication, Wishlist, and Administration

Expected deliverables:

Implement user registration and login.

Implement authentication and role-based authorization.

Build wishlist functionality.

Create admin product and offer management.

Implement outbound click tracking.

Add basic analytics.

End-of-day target: Core user workflows and administrative operations work end-to-end.

DAY 4 — Testing, Deployment, and Documentation

Expected deliverables:

Test the main user journeys and API endpoints.

Fix critical functional and security issues.

Deploy the frontend and backend.

Configure environment variables.

Prepare seed data and demo credentials.

Complete the README and architecture explanation.

Verify that the deployed application works independently of the developer's local machine.

End-of-day target: A working, publicly accessible application with documentation and reproducible setup instructions.



6. MODERATE-TO-HARD ENGINEERING CHALLENGE

Product Normalization and Offer Association

One of the important challenges is associating offers from different merchants with the correct product.

For example, the same watch may appear on different websites under slightly different titles:

“Brand X Watch Model 100”

“Brand X Model 100 Analog Watch”

“Model 100 by Brand X”

The candidate should design a mechanism to associate these offers with one canonical product without incorrectly merging different models, generations, or variants.

The implementation may use a stable product identifier, model number, or a combination of brand, model, and variant attributes.

The candidate must explain:

How canonical products are identified.

How merchant offers are associated with products.

How different variants are kept separate.

How duplicate offers are prevented.

How the system handles missing or inconsistent product information.

A fully automated matching engine is not required. A well-designed data model and reliable association logic are sufficient.

OBJ

7. DATA AND INTEGRATION CONSTRAINTS

The following rules are mandatory.

Do not use paid APIs, paid services, or paid software subscriptions.

Do not use OpenAI, Gemini, or other paid LLM APIs.

Do not require real affiliate accounts or merchant partnerships.

Do not scrape websites by bypassing authentication, CAPTCHAs, access restrictions, or anti-bot protections.

Do not use another developer's complete project or submit an existing template as the finished implementation.

Use original, seeded, or otherwise authorized product data.

Use valid external product URLs where available.

Clearly document any feature that uses simulated data.

Candidates may use free libraries, CSS frameworks, charting libraries, and free-tier hosting or database services, subject to current provider limits.

The core application must not depend on a paid service to function.

OBJ

8. DEPLOYMENT REQUIREMENTS

The application must be deployed and accessible through a live URL.

The candidate must submit:

Live frontend URL.

Backend/API URL, where applicable.

Public GitHub repository URL.

Demo credentials for user and admin accounts.

README with local setup instructions.

.env.example containing required environment variable names without actual secrets.

Seed data or instructions for creating the demo dataset.

A short explanation of the application architecture and major design decisions.

The evaluator must be able to browse products, view offers, use the wishlist, and test the admin workflow without needing the candidate's local environment.

OBJ

9. ACCEPTANCE CRITERIA

The project will be considered functionally complete when:

Users can browse, search, filter, sort, and paginate products.

Product details display the correct associated merchant offers.

The lowest listed price is identified correctly.

Clicking a merchant offer redirects to its configured destination.

Outbound clicks are recorded.

Users can register, log in, and manage their own wishlist.

Unauthorized users cannot access admin-only functionality.

Admins can manage products and merchant offers.

Analytics reflect stored application data.

Invalid inputs and failed requests are handled appropriately.

The deployed application is accessible and its core workflows function.

The repository includes the required documentation.

OBJ

10. EVALUATION CRITERIA — 100 MARKS

OBJ

Evaluation Guidelines

Functional completeness — 25 marks

Evaluate the working product catalog, merchant offers, external redirects, wishlist, admin functionality, and analytics.

Backend architecture and database design — 20 marks

Evaluate schema design, API structure, query handling, validation, pagination, and business logic.

Engineering quality and security — 20 marks

Evaluate authentication, authorization, password security, data isolation, redirect validation, and edge-case handling.

Frontend quality and user experience — 15 marks

Evaluate usability, responsive layouts, state management, loading states, and error handling.

Deployment and reliability — 10 marks

Evaluate the live application, environment configuration, API connectivity, and reliability of the main workflows.

Code quality and documentation — 10 marks

Evaluate project organization, naming conventions, meaningful commits, README quality, architecture explanations, and reproducibility.

OBJ

11. SUBMISSION DEADLINE

Time allowed: 4 days (96 hours) from the official assignment release time.

Deadline: 16th October, 2pm

All required deliverables must be submitted before the deadline.

Candidates should prioritize a functional, secure, and well-documented core application over a large number of incomplete features.

OBJ

12. FINAL EXPECTATION

This assessment is intended to evaluate the candidate's ability to independently design, develop, test, and deploy a realistic full-stack application within a limited timeframe.

The objective is not merely to create a visually attractive interface. The candidate should demonstrate sound engineering judgment, maintainable code, reliable business logic, secure API design, appropriate database modeling, and the ability to explain the implementation.

A smaller application that works correctly and is engineered thoughtfully is preferable to a larger application with incomplete or unreliable functionality., React.js, Node.js
Assessment Duration: 4 Days (96 Hours)

Total Marks: 100
Deployment Requirement: Mandatory
Cost Constraint: Zero-cost implementation using free tools and services

---

1. PROJECT OVERVIEW

Business Context

We are looking to build a product discovery and affiliate commerce platform that allows users to browse products across different categories and compare offers from external e-commerce platforms.

Unlike a traditional e-commerce website, our platform will not directly sell or maintain inventory of products. Instead, it will display product information and redirect users to external merchant websites, such as Amazon or Flipkart, when they choose to purchase a product.

For example, a user visits DealBridge and searches for a watch. The platform displays the watch's image, name, brand, specifications, and available merchant offers. When the user clicks **"Buy Now,"** they are redirected to the corresponding product page on the external merchant's website.

The objective is to develop a simplified but functional version of a real-world product discovery and affiliate commerce platform.

Important: This assessment does not require actual affiliate partnerships, paid APIs, or live merchant integrations. Candidates may use seeded product data and valid external product URLs.

2. PROJECT OBJECTIVES

The application should demonstrate the candidate's ability to:

- Develop a full-stack application using the MERN stack.
- Design and implement RESTful APIs.
- Model and manage relationally connected data using MongoDB.
- Implement authentication and role-based access control.
- Develop product search, filtering, sorting, and pagination.
- Associate multiple merchant offers with a single product.
- Implement secure external redirects and click tracking.
- Build a responsive and user-friendly interface.
- Deploy a working application using free hosting options.

3. FUNCTIONAL REQUIREMENTS

Module 1: Product Discovery and Listing

Develop a responsive homepage displaying products across multiple categories, such as:

- Watches
- Smartphones
- Laptops
- Headphones
- Fashion
- Home appliances

Each product card should display:

- Product image
- Product name
- Brand
- Category
- Starting price or lowest available offer price
- Availability status, where applicable
- A link to view product details

Search and filtering requirements:

- Search products by name, brand, or model.
- Filter products by category and brand.
- Filter products by minimum and maximum price.
- Sort products by price, newest arrivals, and popularity.
- Implement pagination for product listings.

Search and filtering should operate on backend data rather than only filtering products already loaded into the browser.

Module 2: Product Details and Merchant Offers

Each product must have a dedicated details page containing:

- Product name and description
- Brand and model
- Product images
- Specifications or key features
- Category
- Available merchant offers

For example, the same watch may have offers from multiple external stores.

Each merchant offer should contain:

- Merchant name
- Product price
- External product URL
- Availability status, where applicable
- Last-updated timestamp

Business rule: Multiple merchant offers should be associated with one canonical product whenever they represent the same product and variant.

The platform should identify and highlight the lowest available listed price.

Module 3: External Merchant Redirection

When a user clicks **"Buy Now,"** the application should redirect them to the configured external product page.

Requirements:

- Each offer must have a valid destination URL.
- The system should record the product, merchant, timestamp, and click event.
- The click-tracking mechanism should not unnecessarily block the redirect.
- Invalid or untrusted redirect destinations must be rejected.
- Users must not be able to supply arbitrary redirect URLs through request parameters.

The platform does not need to process payments or manage orders.

Module 4: User Authentication and Wishlist

Implement user registration, login, and logout.

Authenticated users should be able to:

- Add products to their wishlist.
- Remove products from their wishlist.
- View their saved products.
- Retain wishlist data after logging out and logging in again.

Access control requirements:

- Users must only be able to view and modify their own wishlist.
- Authentication and authorization must be enforced on the backend.
- Passwords must be securely hashed and must never be stored in plain text.

Module 5: Admin Dashboard

Create a basic administrative interface for managing the product catalog.

Administrators should be able to:

- Add new products.
- Edit existing product details.
- Add or update merchant offers.
- Change product availability or deactivate products.
- View basic merchant click statistics.

Security requirement: Only authorized administrators may create, modify, or deactivate products and merchant offers. Hiding admin controls in the frontend is not sufficient; the backend must enforce permissions.

Module 6: Basic Analytics

Implement a simple dashboard displaying:

- Total number of products
- Total number of merchant offers
- Total outbound merchant clicks
- Most-clicked products
- Most-clicked merchants

Analytics should be calculated using stored application data rather than hardcoded statistics.

---

4. TECHNICAL REQUIREMENTS

Frontend

- React.js
- Responsive UI for desktop and mobile
- React Router or an equivalent routing solution
- Appropriate state management
- API integration with the backend
- Form validation and error handling

- Loading, empty, and error states

Backend

- Node.js and Express.js
- RESTful API architecture
- Authentication and authorization middleware
- Request validation and centralized error handling
- Pagination, filtering, and sorting
- Appropriate HTTP status codes
- Secure handling of user input and external URLs

Database

Use MongoDB to store application data.

Suggested collections:

- Users
- Products
- Merchants
- MerchantOffers
- Wishlists
- ClickEvents

Candidates may modify the schema or collection structure according to their implementation, provided the data model supports the required functionality.

Suggested API Endpoints

The following endpoints are indicative. Candidates may use a different RESTful structure if it is documented and implemented consistently.

Products

- `GET /api/products` — List, search, filter, sort, and paginate products.
- `GET /api/products/:id` — Retrieve product details.
- `POST /api/products` — Create a product (admin only).
- `PUT /api/products/:id` — Update a product (admin only).
- `DELETE /api/products/:id` — Deactivate or remove a product (admin only).

Merchant Offers

- `GET /api/products/:id/offers` — Retrieve offers for a product.

- `POST /api/products/:id/offers` — Add an offer (admin only).
- `PUT /api/offers/:id` — Update an offer (admin only).

Authentication and Wishlist

- `POST /api/auth/register` — Register a user.
- `POST /api/auth/login` — Authenticate a user.
- `GET /api/wishlist` — Retrieve the authenticated user's wishlist.
- `POST /api/wishlist/:productId` — Add a product to the wishlist.
- `DELETE /api/wishlist/:productId` — Remove a product from the wishlist.

Analytics

- `GET /api/admin/analytics` — Retrieve basic product and click statistics (admin only).

Candidates may add further endpoints as needed.

---

5. FOUR-DAY IMPLEMENTATION PLAN

DAY 1 — Frontend and Product Discovery

Expected deliverables:

- Set up the frontend and backend projects.
- Design the homepage and product listing pages.
- Implement product cards and product detail pages.
- Implement search, filtering, sorting, and pagination interfaces.
- Create an initial product dataset.

End-of-day target: Users can browse and search the product catalog through a functional interface.

DAY 2 — Backend, Database, and Merchant Offers

Expected deliverables:

- Design the MongoDB schemas.
- Implement REST APIs for products and merchant offers.
- Connect the frontend to the backend.
- Associate multiple merchant offers with a canonical product.
- Implement lowest-price selection and server-side filtering.

- Add validation and error handling.

End-of-day target: Product information and merchant offers are loaded from the database and displayed correctly.

DAY 3 — Authentication, Wishlist, and Administration

Expected deliverables:

- Implement user registration and login.
- Implement authentication and role-based authorization.
- Build wishlist functionality.
- Create admin product and offer management.
- Implement outbound click tracking.
- Add basic analytics.

End-of-day target: Core user workflows and administrative operations work end-to-end.

DAY 4 — Testing, Deployment, and Documentation

Expected deliverables:

- Test the main user journeys and API endpoints.
- Fix critical functional and security issues.
- Deploy the frontend and backend.
- Configure environment variables.
- Prepare seed data and demo credentials.
- Complete the README and architecture explanation.
- Verify that the deployed application works independently of the developer's local machine.

End-of-day target: A working, publicly accessible application with documentation and reproducible setup instructions.

---

6. MODERATE-TO-HARD ENGINEERING CHALLENGE

Product Normalization and Offer Association

One of the important challenges is associating offers from different merchants with the correct product.

For example, the same watch may appear on different websites under slightly different titles:

- “Brand X Watch Model 100”
- “Brand X Model 100 Analog Watch”
- “Model 100 by Brand X”

The candidate should design a mechanism to associate these offers with one canonical product without incorrectly merging different models, generations, or variants.

The implementation may use a stable product identifier, model number, or a combination of brand, model, and variant attributes.

The candidate must explain:

1. How canonical products are identified.
2. How merchant offers are associated with products.
3. How different variants are kept separate.
4. How duplicate offers are prevented.
5. How the system handles missing or inconsistent product information.

A fully automated matching engine is not required. A well-designed data model and reliable association logic are sufficient.

---

7. DATA AND INTEGRATION CONSTRAINTS

The following rules are mandatory.

- Do not use paid APIs, paid services, or paid software subscriptions.
- Do not use OpenAI, Gemini, or other paid LLM APIs.
- Do not require real affiliate accounts or merchant partnerships.
- Do not scrape websites by bypassing authentication, CAPTCHAs, access restrictions, or anti-bot protections.
- Do not use another developer's complete project or submit an existing template as the finished implementation.
- Use original, seeded, or otherwise authorized product data.
- Use valid external product URLs where available.
- Clearly document any feature that uses simulated data.

Candidates may use free libraries, CSS frameworks, charting libraries, and free-tier hosting or database services, subject to current provider limits.

The core application must not depend on a paid service to function.

---

8. DEPLOYMENT REQUIREMENTS

The application must be deployed and accessible through a live URL.

The candidate must submit:

1. Live frontend URL.
2. Backend/API URL, where applicable.
3. Public GitHub repository URL.
4. Demo credentials for user and admin accounts.
5. README with local setup instructions.
6. `.env.example` containing required environment variable names without actual secrets.
7. Seed data or instructions for creating the demo dataset.
8. A short explanation of the application architecture and major design decisions.

The evaluator must be able to browse products, view offers, use the wishlist, and test the admin workflow without needing the candidate's local environment.

---

9. ACCEPTANCE CRITERIA

The project will be considered functionally complete when:

- Users can browse, search, filter, sort, and paginate products.
- Product details display the correct associated merchant offers.
- The lowest listed price is identified correctly.
- Clicking a merchant offer redirects to its configured destination.
- Outbound clicks are recorded.
- Users can register, log in, and manage their own wishlist.
- Unauthorized users cannot access admin-only functionality.
- Admins can manage products and merchant offers.
- Analytics reflect stored application data.
- Invalid inputs and failed requests are handled appropriately.
- The deployed application is accessible and its core workflows function.
- The repository includes the required documentation.

---

10. EVALUATION CRITERIA — 100 MARKS

Evaluation Area	Marks
Functional completeness	25
Backend architecture and database design	20
Engineering quality and security	20
Frontend quality and user experience	15
Deployment and reliability	10
Code quality and documentation	10
TOTAL	**100**

Evaluation Guidelines

Functional completeness — 25 marks

Evaluate the working product catalog, merchant offers, external redirects, wishlist, admin functionality, and analytics.

Backend architecture and database design — 20 marks

Evaluate schema design, API structure, query handling, validation, pagination, and business logic.

Engineering quality and security — 20 marks

Evaluate authentication, authorization, password security, data isolation, redirect validation, and edge-case handling.

Frontend quality and user experience — 15 marks

Evaluate usability, responsive layouts, state management, loading states, and error handling.

Deployment and reliability — 10 marks

Evaluate the live application, environment configuration, API connectivity, and reliability of the main workflows.

Code quality and documentation — 10 marks

Evaluate project organization, naming conventions, meaningful commits, README quality, architecture explanations, and reproducibility.

---

11. SUBMISSION DEADLINE

Time allowed: 4 days (96 hours) from the official assignment release time.

Deadline: 16th October, 2pm

All required deliverables must be submitted before the deadline.

Candidates should prioritize a functional, secure, and well-documented core application over a large number of incomplete features.

---

12. FINAL EXPECTATION

This assessment is intended to evaluate the candidate's ability to independently design, develop, test, and deploy a realistic full-stack application within a limited timeframe.

The objective is not merely to create a visually attractive interface. The candidate should demonstrate sound engineering judgment, maintainable code, reliable business logic, secure API design, appropriate database modeling, and the ability to explain the implementation.

A smaller application that works correctly and is engineered thoughtfully is preferable to a larger application with incomplete or unreliable functionality.